Neural Networks 206 (2027) 109807

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

Directed homophily-aware graph neural network ⋆

Aihu Zhang [a,*], Jiaxing Xu [a], Mengcheng Lan [a], Shili Xiang [b], Yiping Ke [a]

[a] College of Computing and Data Science, Nanyang Technological University, 50 Nanyang Avenue, 639798, Singapore
[b] Institute for Infocomm Research, A*STAR, 2 Fusionopolis Way, Innovis 08-01, 138634, Singapore

ARTICLE INFO

Keywords:
Graph neural networks
Heterophily
Directed graph

ABSTRACT

Graph Neural Networks (GNNs) have achieved significant success in various learning tasks on graph-structured data. Nevertheless, most GNNs struggle to generalize to heterophilic neighborhoods. Additionally, many GNNs ignore the directional nature of real-world graphs, resulting in suboptimal performance on directed graphs with asymmetric structures. In this work, we propose Directed Homophily-aware Graph Neural Network (DHGNN), a novel framework that addresses these limitations by incorporating homophily-aware and direction-sensitive components. DHGNN employs a resettable gating mechanism to adaptively modulate message contributions based on homophily levels and informativeness, and a structure-aware noise-tolerant fusion module to effectively integrate node representations from the original and reverse directions. Extensive experiments on both homophilic and heterophilic directed graph datasets demonstrate that DHGNN outperforms state-of-the-art methods in node classification and link prediction. In particular, DHGNN improves over the best baseline by up to 15.07% in link prediction. Our analysis further shows that the gating mechanism captures directional homophily gaps and fluctuating homophily across layers, providing deeper insights into message-passing behavior on complex graph structures.

1. Introduction

Graph Neural Networks (GNNs) (Hamilton et al., 2017; Kipf & Welling, 2016; Scarselli et al., 2009; Veličković et al., 2017) are powerful models for learning graph representations, typically following a message passing paradigm (Gilmer et al., 2017), where nodes iteratively aggregate and combine messages from their neighbors. After several iterations, each node obtains an embedding that captures information from itself and nearby nodes, supporting various node- and graph-level tasks (Khemani et al., 2024). Despite their success, message-passing GNNs often struggle to generalize in two settings: (1) heterophilic neighborhoods, and (2) directed graphs.

Heterophily refers to the case where neighboring nodes have dissimilar features or labels (Guo et al., 2025; Li et al., 2022; Tang et al., 2023; Xie et al., 2025; Zhu et al., 2020a). The node-wise homophily ratio (Pei et al., 2020) quantifies the proportion of a node's neighbors sharing the same label, which is commonly used to measure heterophily. GNNs often perform poorly on graphs with low average homophily (Zhu et al., 2020b), partly due to over-smoothing (Li et al., 2018), where deeper models produce indistinguishable node representations. Several methods have been proposed to address heterophilic settings and mitigate over-smoothing (He et al., 2022; Luan et al., 2022; Song et al., 2023; Suresh et al., 2021; Yan et al., 2022; Zhu et al., 2020b). However, these approaches either fail to capture fluctuations in homophily across different hops (Song et al., 2023; Zhu et al., 2020b) (illustrated in Fig. 1a) or lack the ability to make personalized, node-specific decisions (He et al., 2022; Luan et al., 2022; Suresh et al., 2021; Yan et al., 2022), limiting their effectiveness in complex neighborhood structures.

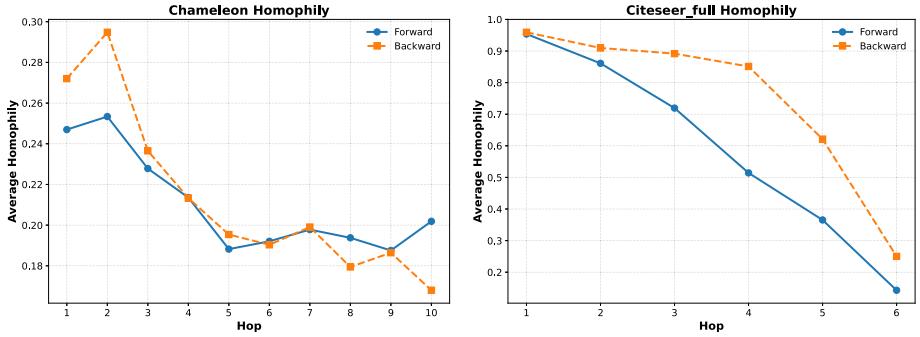
Edge directionality is another important source of information that GNNs fail to fully exploit. While the traditional approach of converting directed graphs to undirected ones (Hamilton et al., 2017; Kipf & Welling, 2016; Veličković et al., 2017) preserves intra-class relationships, it imposes artificial symmetry that can obscure important directional, inter-class information, as illustrated in Fig. 2. Both forward (original) and backward (reversed) directions can contain complementary and informative signals that are critical for understanding complex graph structures, such as inter-domain connections in citation networks. Encoding the two directions separately demonstrates effectiveness (Huang et al., 2024b; Rossi et al., 2024; Tong et al., 2020b). Nonetheless, these approaches still face limitations. They ignore the homophily and structure differences between neighborhoods in opposite directions, as shown in Fig. 1b. Moreover, their reliance on layer-wise fusion restricts the model's ability to capture long-range dependencies across the graph (Huang et al., 2024b; Rossi et al., 2024).

In this paper, we propose Directed Homophily-aware Graph Neural Network (DHGNN), a novel framework designed to extract meaningful

⋆ Our code is available at: https://anonymous.4open.science/r/DHGNN-7FE5.

* Corresponding author.
E-mail address: zhan0547@e.ntu.edu.sg (A. Zhang).

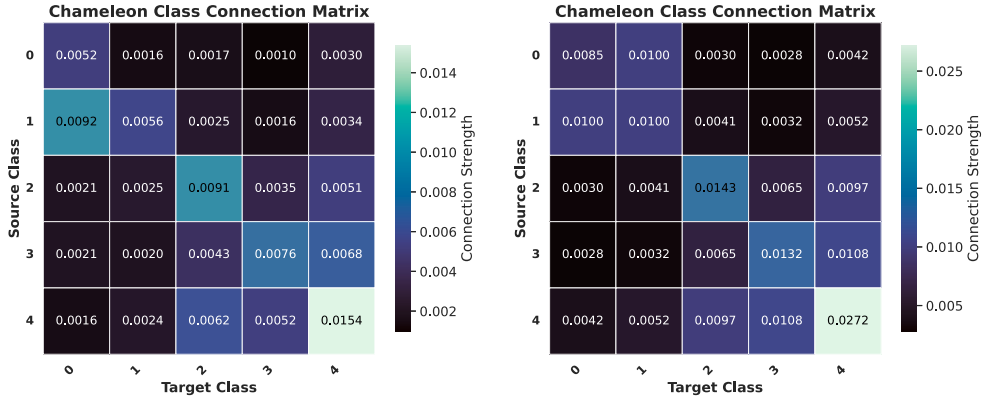
https://doi.org/10.1016/j.neunet.2026.109807
Received 4 January 2026; Received in revised form 15 July 2026; Accepted 26 October 2026
Available online 28 October 2026
0893-6080/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.



(a) The homophily fluctuation on Chameleon.

(b) The directional homophily difference on Citeseer-Full.

Fig. 1. The average node-wise homophily ratio in different hops. Forward denotes the direction in which the graph is constructed while backward denotes the reverse direction. The line chart reveals (a) the homophily ratio does not consistently decrease with increasing distance, and (b) the homophily levels of forward and backward neighbors can be different. We further explore this phenomenon with an example in Appendix A.



(a) Directed Chameleon dataset.

(b) Undirected Chameleon dataset.

Fig. 2. Class connection matrices of the directed and undirected versions of the same graph. The matrices tend to be asymmetric for heterophilic graphs, such as Chameleon. Converting this graph from directed to undirected could eliminate the asymmetry between the edge ratios from class 1 to class 0 and from class 0 to class 1, while the two ratios differ significantly in the original directed graph.

information from complex, noisy neighborhoods by explicitly modeling homophily fluctuations across hops, and structural as well as homophily gaps between opposite directions.

To address the challenges posed by heterophilic neighborhoods, we introduce a homophily-aware gating mechanism that dynamically adjusts gating values for message-passing based on the embedding similarity and informativeness of neighboring nodes across different hops. This design enables the model to handle fluctuating homophily patterns and selectively incorporate relevant signals, even from distant neighbors.

To capture the structural and homophily differences between opposite directions, we design a dual-encoder architecture that separately processes the forward and backward directions of directed graphs. A structure-aware, noise-tolerant fusion mechanism then adaptively integrates the directional embeddings, guided by both node-level features and structural context, to form a unified and expressive node representation. By employing independent encoders along with branch-specific auxiliary losses, the model effectively captures distinct and meaningful information from each direction.

To our knowledge, DHGNN is the first approach to simultaneously address both directional and heterophilic challenges through an adaptive design. Existing spatial methods only partially resolve these issues. For instance, Dir-GNN (Rossi et al., 2024) relies on shared directional weights across all layers, inherently neglecting how the neighborhood informativeness of each direction changes with depth. While NDDGNN (Huang et al., 2024b) improves upon this with adaptive directional weights, it still applies shared message weights across hops, failing to capture homophily fluctuations. Motivated by these limitations, DHGNN is designed to explicitly model both directional structural gaps and cross-hop homophily variations, ensuring a more dynamic and balanced aggregation of the graph topology.

To validate the effectiveness of DHGNN, we conduct comprehensive experiments on five benchmark datasets with varying levels of homophily and directionality. DHGNN is able to outperform state-of-the-art baselines in node classification and link prediction tasks. Further analysis demonstrates its ability to adaptively respond to both the homophily fluctuations across hops and the directional homophily gaps. An analysis of DHGNN's behavior at deeper layers suggests that the model effectively alleviates the over-smoothing issue.

Our key contributions can be summarized as follows:

- We propose a homophily-aware gating mechanism that adaptively regulates message-passing weights across layers and directions, which is mathematically formulated in Eqs. (5) and (6) with non-monotonicity proven in Theorem 1.
- We develop a structure-aware, noise-tolerant fusion strategy that integrates directional embeddings into a unified representation, as defined by the fusion process in Eqs. (7) and (8).
- We validate the proposed DHGNN on five heterophilic and homophilic datasets. The results demonstrate improved classification and link prediction performance by up to 15.07%, along with interpretable gate behavior.

2. Related work

Supervised methods for heterophilic graphs. Various methods have been developed to handle heterophily in graphs. LINKX (Lim et al., 2021) avoids aggregating over heterophilic neighborhoods by separately processing node features and adjacency information. Other approaches decompose ego and neighbor representations or utilize signal separation techniques (He et al., 2022; Li et al., 2022; Luan et al., 2022; Suresh et al., 2021; Zhu et al., 2020b). For example, H2GCN (Zhu et al., 2020b) concatenates information from different neighborhood ranges while excluding self-loops. WRGNN (Suresh et al., 2021) and BM-GCN (He et al., 2022) apply different transformation matrices to ego and neighbor messages. GloGNN (Li et al., 2022) uses both low- and high-pass filters, while ACM-GCN (Luan et al., 2022) distinguishes ego embeddings using identity filters at the channel level.

Several methods employ adaptive weighting strategies. GPR-GNN (Chien et al., 2020) learns Generalized PageRank weights across layers. GGCN (Yan et al., 2022) uses learnable scalars for self and signed neighbor edges. Gradient Gating (G^2) (Rusch et al., 2022) introduces multi-rate gradient flow, and Ordered GNN (Song et al., 2023) applies soft gating to gradually freeze node embeddings and mitigate over-smoothing. More recently, Flow2GNN (Huang et al., 2024a) introduces an operator to disentangle topological information inflow and outflow between connected nodes, and utilizes a similarity-based aggregation model to adjust the flow between homophilic and heterophilic attributes. Furthermore, IGNN (Gu et al., 2024) decomposes message passing into Separative Neighborhood Transformation (SN), Inceptive Neighborhood Aggregation (IN), and Neighborhood Relationship Learning (NR), and explores variant adaptive strategies in these components to accommodate varying levels of homophily. Most gating-based approaches either handle the embeddings from each layer independently, or assume a monotonically decreasing homophily with distance, overlooking potential fluctuations, which may limit the ability to extract informative signals from distant nodes. To overcome the limitations of node-level aggregations under severe heterophily, recent studies emphasize explicitly modeling edge distributions and higher-order structures. For instance, H_2OGNN (Wang et al., 2025) reveals that node-level attribute inference is susceptible to edge distribution shifts, and instead frames edge identification as an Out-of-Distribution (OOD) detection problem. Concurrently, the exploration of heterophily has expanded to hypergraphs, where HyperUFG (Li et al., 2025) employs dual low-pass and high-pass spectral filters to capture multi-scale interactions. In contrast to these methods that focus on edge-level OOD detection or hypergraph topology, our DHGNN tackles complex graph heterophily by modeling cross-hop homophily fluctuations and structural directionality through a resettable gating mechanism.

Self-supervised methods for heterophilic graphs. Beyond these supervised architectures, complementary paradigms address heterophily through self-supervised learning and parameter-efficient fine-tuning. PolyGCL (Chen et al., 2024) extends graph contrastive learning to heterophilic settings by adaptively capturing high-frequency signals via learnable spectral polynomial filters. UniPrompt (Huang et al., 2025) tackles the homophily-to-heterophily distribution shift in pre-trained models by constructing a learnable, task-specific prompt graph at the input level. These frameworks offer orthogonal insights into structural adaptation, complementing supervised adaptive message-passing designs.

Directed Graph Neural Networks. Traditional GNNs often convert directed graphs into undirected ones, potentially disrupting important inter-class links. To address this limitation, various methods have been developed from both spectral and spatial perspectives to explicitly model directionality in graph learning. Spectral approaches include DiGCN (Tong et al., 2020a), which approximates the Laplacian using personalized PageRank transition probabilities, and MagNet (Zhang et al., 2021), which introduces a Magnetic Laplacian with complex-valued phase matrices to encode directionality. Building on this, Geisler et al. (2023) uses both the Magnetic Laplacian and directional random walk encodings as positional encodings for Transformers, while Huang et al. (2024c) further enhances the Magnetic Laplacian-based encoding with additional structural factors. However, these methods often apply the same transformation to both edge directions, overlooking directional differences in neighborhood homophily. Spatial approaches explicitly separate directional representations. DGCN (Tong et al., 2020b) uses distinct functions for first- and second-order proximity. Dir-GNN (Rossi et al., 2024) independently encodes each direction and combines them with manually set fusion weights, while NDDGNN (Huang et al., 2024b) adaptively learns fusion weights based on structural and feature similarities. Although effective for local interactions, such layer-wise fusion may struggle to capture long-range dependencies along extended paths. Moreover, effectively fusing diverse structural signals remains a broader challenge in complex networks. Recent advancements in related domains, such as adaptive multi-basis spectral filtering (Wang & Liu, 2025) and robust data clustering via sparse coding (Shi & Liu, 2023) or interpretable neural structures (Gai & Liu, 2025), highlight the importance of noise-tolerant representation learning. Inspired by these principles, DHGNN introduces a structure-aware, noise-tolerant directional fusion mechanism to robustly integrate forward and backward dependencies.

3. Preliminaries

Directed graphs. In this paper, we address the tasks of semi-supervised node classification and link prediction on attributed directed graphs. Given a directed graph $\mathcal{G} = (\mathcal{V}, \mathbf{X}, \mathbf{A})$ with n nodes and m edges, each node $i \in \mathcal{V}$ is associated with a feature vector, forming a feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$, where d_{in} is the feature dimension. The graph also includes a label $y_i \in \{1, \ldots, C\}$ for each node corresponding to C classes. The directed structure is represented by an adjacency matrix $\mathbf{A} \in \{0,1\}^{n \times n}$, where $\mathbf{a}_{ij} = 1$ if there is a directed edge from node i to node j, and zero otherwise.

Message-passing neural networks. Most GNNs follow a message passing mechanism (Xu et al., 2018) iteratively updating representations of node v, namely $\mathbf{h}_v^{(l)} \in \mathbb{R}^p$, where l denotes the layer number and p is the hidden dimension at the current layer. Each layer of a message passing GNN comprises two fundamental steps, i.e., aggregation and combination, which can be formulated as:

$$\begin{aligned} \mathbf{m}_v^{(l)} &= \text{AGGREGATE}^{(l)}(\{\mathbf{h}_u^{(l-1)} | u \in \mathcal{N}(v)\}), \\ \mathbf{h}_v^{(l)} &= \text{COMBINE}^{(l)}(\mathbf{h}_v^{(l-1)}, \mathbf{m}_v^{(l)}), \end{aligned} \tag{1}$$

where the AGGREGATE$(\cdot)$ takes the embeddings of nodes in the neighborhood $\mathcal{N}(v)$ of a node v and generates a message $\mathbf{m}_v^{(l)}$, and COMBINE$(\cdot)$ incorporates the message with the previous node embedding $\mathbf{h}_v^{(l-1)}$ to produce the updated embedding $\mathbf{h}_v^{(l)}$. The initial embedding $\mathbf{h}_v^{(0)}$ is set to be the original node feature $\mathbf{x}_v$ of node v.

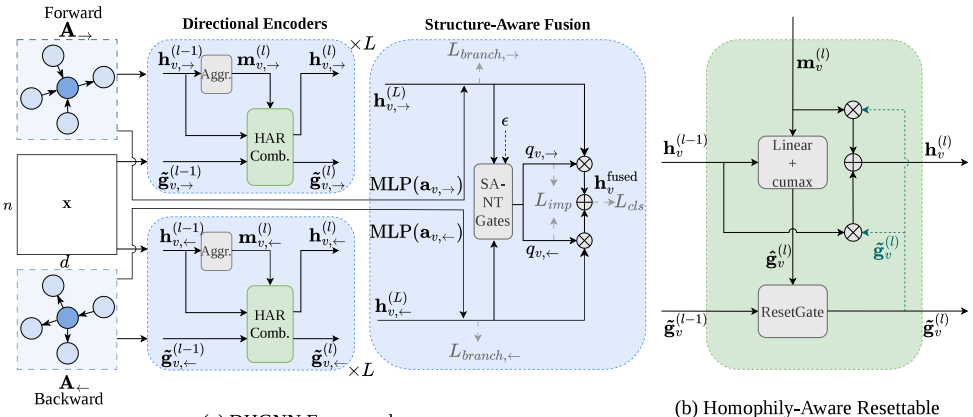
Homophily. Homophily is the tendency of connected nodes in a graph to share similar labels or features, and it underpins the success of many GNNs. A common way to measure this is the node-wise homophily ratio, defined for a node v with label y_v as

$$h_v = \frac{\left|\left\{u \in \mathcal{N}(v) \mid y_u = y_v\right\}\right|}{|\mathcal{N}(v)|}, \tag{2}$$

where $\mathcal{N}(v)$ is the set of neighbors of v.

Directional Homophily. We extend the traditional definition to directed graphs by explicitly differentiating between edge directions. In the context of GNN with a directed graph, we focus on the influence of the in-neighbors. Let $\mathcal{N}^-(v)$ represent the set of in-neighbors of a node v. We define the directional homophily ratios as:

$$h_v^- = \frac{|\{u \in \mathcal{N}^-(v) \mid y_u = y_v\}|}{|\mathcal{N}^-(v)|}. \tag{3}$$



(a) DHGNN Framework.

(b) Homophily-Aware Resettable (HAR) Combination.

Fig. 3. The framework of DHGNN. "SA-NT Gates" is short for the structure-aware noise-tolerant gates. The model consists of two message-passing modules (one for each edge direction) and a fusion module. Node representations are learned independently for the forward and backward directions, and subsequently fused into a unified representation.

By directional homophily gap, we refer to the forward and backward versions of a raw directed graph, and calculate the difference of their directional homophily ratios, each with in-neighbors used as above. Essentially, it is equivalent to calculating the homophily difference on the raw directed graph by contrasting in- and out-neighbors, namely $|h_v^- - h_v^+|$, where h_v^+ is the homophily ratio calculated with out-neighbors.

Multi-hop Homophily. To further capture homophily fluctuations across hops, the definition of directional homophily ratio can be extended to k-hop in-neighbors by substituting $\mathcal{N}^-(v)$ with $\mathcal{N}^{(k)-}(v)$. We define this k-hop neighborhood as the set of nodes whose shortest incoming path to v has exactly k steps. The k-hop directional homophily ratio can be expressed as:

$$h_v^{(k)-} = \frac{\left|\left\{u \in \mathcal{N}^{(k)-}(v) \;\middle|\; y_u = y_v\right\}\right|}{\left|\mathcal{N}^{(k)-}(v)\right|}. \tag{4}$$

The overall graph homophily is computed by averaging the homophily ratio over all nodes. High homophily typically benefits standard GNNs, while low homophily (high heterophily) poses challenges for conventional message passing.

In scenarios where node labels are partially or entirely unavailable, such as in link prediction tasks, the notion of homophily can be extended to quantify feature similarity between neighboring nodes.

4. Methodology

4.1. Model architecture

To enable the model to effectively handle varying levels of homophily in the forward and backward directions, we propose a novel and effective framework specifically designed to disentangle and adaptively integrate directional information, namely DHGNN as shown in Fig. 3a.

The model employs two parallel encoders based on a resettable gated message-passing scheme. The two encoders share the same architecture but with independent parameters, which are responsible for processing forward and backward edges, respectively. Within each encoder, a gating vector dynamically regulates the contribution of incoming messages. The resettable gating mechanism is designed to attenuate or amplify messages based on their informativeness, particularly preserving informative long-range dependencies. This enables the model to propagate relevant information more effectively across multiple hops.

The directional embeddings are then combined using a structure-aware fusion mechanism. Fusion gating values are derived from both directional embeddings and structural cues from the directed adjacency matrices, allowing the model to integrate directional information based on graph topology.

To avoid the collapse of the two directional encoders into a single dominant pathway, DHGNN introduces an auxiliary importance loss that discourages extreme imbalance in the fusion decision scores. Furthermore, to encourage disentanglement and minimize redundancy between the forward and backward encoders, a branch classification loss is incorporated during training.

4.2. Homophily-aware gated message passing

As presented in the previous sections, the homophily ratio could vary in the backward and forward directions, and the homophily ratio is not strictly decreasing in all datasets.

Therefore, to capture fluctuations in local homophily and gaps between forward and backward directions simultaneously, we introduce a homophily-aware gated message-passing mechanism, as defined in Eq. ((5)). In each aggregation layer, a separate gating vector is updated with a homophily-aware resettable (HAR) combination module alongside the node embeddings (shown in Fig. 3b), modulating the influence of forward messages from the neighborhood in a node-specific manner. For notational clarity, we present the message-passing equations for a generic encoder branch, omitting the explicit direction index $d \in \{\leftarrow, \rightarrow\}$; the complete dual-directional procedure is detailed in Algorithm 1.

$$\begin{aligned}
\mathbf{m}_v^{(l)} &= \text{mean}\left(\left\{\mathbf{h}_u^{(l-1)} | u \in \mathcal{N}(v)\right\}\right), \\
\hat{\mathbf{g}}_v^{(l)} &= \text{cumax}_{\rightarrow}\left(\mathbf{W}^{(l)}\left[\mathbf{h}_v^{(l-1)} || \mathbf{m}_v^{(l)}\right] + \mathbf{b}^{(l)}\right), \\
\tilde{\mathbf{g}}_v^{(l)} &= \text{ResetGate}(\tilde{\mathbf{g}}_v^{(l-1)}, \hat{\mathbf{g}}_v^{(l)}), \\
\mathbf{h}_v^{(l)} &= \tilde{\mathbf{g}}_v^{(l)} \circ \mathbf{h}_v^{(l-1)} + (1 - \tilde{\mathbf{g}}_v^{(l)}) \circ \mathbf{m}_v^{(l)},
\end{aligned} \tag{5}$$

where $\text{cumax}_{\rightarrow}(\cdot) = \text{cumsum}_{\rightarrow}(\text{softmax}(\cdot))$ denotes a composite operation that applies the softmax function followed by a cumulative sum. The subscript $\rightarrow$ indicates that cumulative summation operates along feature coordinates in a left-to-right direction, distinguishing it from the directional graph indicators. $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are the learnable weight and bias term in the multilayer perceptron (MLP) layer, $||$ stands for concatenation, and $\circ$ denotes element-wise multiplication. The first equation represents the aggregation step in the message-passing layer, while the remaining three equations define the combination step.

The gating vector is updated using a Gated Recurrent Unit (GRU)-inspired resettable gating mechanism, which enables selective resetting of its values. The reset gate $\mathbf{r}_v^{(l)}$ is computed via an MLP conditioned on the current preliminary gating vector $\hat{\mathbf{g}}_v^{(l)}$ and the previous gating vector

$\tilde{\mathbf{g}}_v^{(l-1)}$:

$$\begin{aligned}\tilde{\mathbf{g}}_v^{(l)} &= \text{ResetGate}(\tilde{\mathbf{g}}_v^{(l-1)}, \hat{\mathbf{g}}_v^{(l)}) = \left(1 - \mathbf{r}_v^{(l)}\right) \circ \tilde{\mathbf{g}}_v^{(l-1)} + \mathbf{r}_v^{(l)} \circ \hat{\mathbf{g}}_v^{(l)}, \\ \mathbf{r}_v^{(l)} &= \sigma(\mathbf{W}_r \hat{\mathbf{g}}_v^{(l)} + \mathbf{U}_r \tilde{\mathbf{g}}_v^{(l-1)} + \mathbf{b}_r),\end{aligned} \tag{6}$$

where $\sigma(\cdot)$ stands for the sigmoid function, and $\mathbf{W}_r$, $\mathbf{U}_r$ and $\mathbf{b}_r$ are the learnable weights and the bias term, respectively. The values in the gating vector tend to increase when the forward neighborhood messages deviate from the current node embedding. These values are constrained within the range $[0, 1]$, where higher values correspond to a reduced influence from neighborhood messages at the current layer. This selective suppression of information from heterophilic neighbors helps mitigate the over-smoothing issue commonly observed in heterophilic graphs.

While the gating strategy at the combination step is inspired by Ordered GNN (Song et al., 2023), the original design assumes that the neighborhood homophily ratio strictly decreases with increasing hop distance. In contrast, our gating mechanism is designed to accommodate fluctuations in neighborhood homophily across different hops. To this end, the gating vector values are not constrained to follow a strictly monotonic trend across layers. This behavior is enabled by the resettable gating mechanism, which allows gating values to increase when the homophily ratio we infer from the embedding similarity decreases or when the incoming message is uninformative, and decrease otherwise. Such flexibility enables the model to gradually attenuate less relevant signals from distant nodes, while still retaining useful long-range information when necessary. As a result, DHGNN can dynamically adapt its receptive field depending on the directionality and structure of the graph, leading to more expressive and context-aware node representations. We provide an analysis of the monotonicity of gating values in Appendix B.

To prevent premature saturation of gating vectors during message passing and reduce the parameter count in the gating network, we employ a chunking strategy similar to the one in Song et al. (2023).

4.3. Structure-aware noise-tolerant fusion

The representations learned from the forward and backward directions can differ significantly due to variations in neighborhood homophily and linking patterns. To fully exploit the information provided by each direction, we design a structure-aware noise-tolerant fusion mechanism. This mechanism integrates the directional node embeddings generated by two separate encoders, producing a unified representation before it is passed to downstream tasks. To decouple the optimization of the directional encoders from the fusion mechanism, we detach the final directional embeddings before fusion: $\bar{\mathbf{h}}_{v,d}^{(L)} = \text{StopGrad}(\mathbf{h}_{v,d}^{(L)})$ for $d \in \{\leftarrow, \rightarrow\}$. The fusion process is then formulated as follows:

$$\mathbf{h}_v^{\text{fused}} = \text{fuse}\left(q_{v,\leftarrow} \bar{\mathbf{h}}_{v,\leftarrow}^{(L)} + q_{v,\rightarrow} \bar{\mathbf{h}}_{v,\rightarrow}^{(L)}\right), \tag{7}$$

where $\text{fuse}(\cdot)$ is a linear layer, scalars $q_{v,\leftarrow}$ and $q_{v,\rightarrow}$ indicate the decision scores for each direction, and L denotes the number of encoder layers. These scores are given by a noise-tolerant MLP-based gating mechanism (To simplify the notation, we omit the layer index L and use only a single-direction subscript in the following equations, as the processing steps for the two directions are analogous.):

$$\begin{aligned}q_{v,\leftarrow}, q_{v,\rightarrow} &= \text{Softmax}\left(\text{NTB}(\tilde{\mathbf{h}}_{v,\leftarrow}), \text{NTB}(\tilde{\mathbf{h}}_{v,\rightarrow})\right), \\ \tilde{\mathbf{h}}_{v,d} &= \bar{\mathbf{h}}_{v,d}^{(L)} + \beta \text{MLP}(\mathbf{a}_{v,d}), \quad d \in \{\leftarrow, \rightarrow\}, \\ \text{NTB}(\tilde{\mathbf{h}}_{v,d}) &= \tilde{\mathbf{h}}_{v,d} \mathbf{W}_{1,d} + \epsilon \cdot \text{Softplus}(\tilde{\mathbf{h}}_{v,d} \mathbf{W}_{2,d}),\end{aligned} \tag{8}$$

where $\text{NTB}(\cdot)$ stands for the noise-tolerant block, $\mathbf{a}_{v,d}$ denotes the structural neighborhood vector of node v extracted from the directional adjacency matrix $\mathbf{A}_d$, and $\epsilon \sim N(0, 1)$ denotes standard Gaussian noise. The incorporation of Gaussian noise in the gating mechanism follows successful practices in established MoE methodologies, such as Shazeer et al. (2017). In our DHGNN, this component brings two benefits: more balanced utilization of two directional encoders; and more exploration in early training. Adding tunable, per-component Gaussian noise ensures exploratory gradient propagation through the fusion gating parameters, especially in the early stages of learning, and helps avoid extreme polarization of directional weights. $\mathbf{W}_{1,\leftarrow}, \mathbf{W}_{2,\leftarrow} \in \mathbb{R}^{p \times 1}$ are learnable weights that control clean and noisy scores, respectively. Since the adjacency matrix of a directed graph is asymmetric, it can provide richer structural information than its undirected counterpart. To fully leverage this asymmetric structure in guiding the fusion process, we incorporate an adjacency-based embedding, which has been proven effective as a structural embedding in LINKX (Lim et al., 2021), into the input of $\text{NTB}(\cdot)$. This integration is governed by the hyperparameter β.

To prevent two encoders from collapsing into one, we incorporate an importance loss following Wang et al. (2023):

$$\begin{aligned}\text{Importance}(\bar{\mathbf{h}}_{\leftarrow}) &= \sum_{v \in \mathcal{V}} q_{v,\leftarrow}, \\ \text{Importance}(\bar{\mathbf{h}}_{\rightarrow}) &= \sum_{v \in \mathcal{V}} q_{v,\rightarrow}, \\ L_{imp} &= \text{CV}(\text{Importance}(\bar{\mathbf{h}}_{\leftarrow}), \text{Importance}(\bar{\mathbf{h}}_{\rightarrow}))^2,\end{aligned} \tag{9}$$

where the importance score, $\text{Importance}(\cdot)$, is defined as the sum of decision scores $q_{v,\leftarrow}$ and $q_{v,\rightarrow}$ across all nodes in the same direction. Here, $\text{CV}(\cdot)$ denotes the coefficient of variation. Consequently, the importance loss L_{imp} quantifies the variability of these importance scores, encouraging all encoders to maintain a comparable level of importance.

4.4. Objective function

The final objective function consists of three components. The primary component is the classification loss L_{cls}, computed using the fused node embedding $\mathbf{h}_v^{\text{fused}}$. We adopt focal loss for node classification and cross-entropy loss for link prediction.

The first auxiliary component is the importance loss L_{imp}, as described in the previous section. The second auxiliary component, the branch classification loss L_{branch}, encourages each branch to independently learn information useful for node classification. It is calculated as the average of classification losses from each directional branch. Formally, $L_{branch} = (L_{branch,\leftarrow} + L_{branch,\rightarrow})/2$. We adopt cross-entropy loss for both directions.

To prevent the fusion mechanism from interfering with the individual optimization of each directional encoder, we apply stop-gradient operations to the directional embeddings after computing their respective losses and before they are fused into the final output. The stop-gradient operation effectively prevents gradients from propagating back through the encoder for each branch, ensuring that the previous quantities (such as the node embeddings from each encoder) are treated as fixed targets rather than learnable inputs during the update step. The two auxiliary losses are weighted by hyperparameters λ_1 and λ_2, respectively. λ_2 is introduced to form the total loss as a convex combination of the losses from the fused output and the individual branches. Thus, the overall objective function is formulated as:

$$L_{total} = (1 - \lambda_2)(L_{cls} + \lambda_1 L_{imp}) + \lambda_2 L_{branch}. \tag{10}$$

The overall procedure, encompassing the forward pass and training of DHGNN, is summarized in Algorithm 1.

5. Experiments

5.1. Datasets

We evaluate our model on five representative benchmarks. Cora-ML and Citeseer-Full (Bojchevski & Günnemann, 2017) are homophilic citation networks whose nodes represent scientific publications with category annotations and edges denote citation links. Chameleon and Squirrel (Pei et al., 2020) are heterophilic Wikipedia page–page networks. While originally derived from mutual links, the standard benchmark versions used here represent the intrinsic, directed hyperlinks between articles, equipped with bag-of-nouns features and traffic-based labels. Roman-Empire (Platonov et al., 2023) is a word-level heterophilic

Algorithm 1 Forward pass and training of DHGNN.

Input: Graph $\mathcal{G} = (\mathcal{V}, \mathbf{X}, \mathbf{A})$, directional adjacency matrices $\mathbf{A}_{\leftarrow}, \mathbf{A}_{\rightarrow}$, labels $\mathbf{y}$, number of layers L, hyperparameters $\lambda_1, \lambda_2, \beta$
Output: Trained model parameters

Initialize node embeddings $\mathbf{h}_{v,\leftarrow}^{(0)} = \mathbf{h}_{v,\rightarrow}^{(0)} = \mathbf{X}_v, \forall v \in \mathcal{V}$
for $l = 1, 2, \ldots, L$ **do**
  for direction $d \in \{\leftarrow, \rightarrow\}$ **do**
    for $v \in \mathcal{V}$ **do**
      $\mathbf{m}_{v,d}^{(l)} = \text{mean}(\{\mathbf{h}_{u,d}^{(l-1)} \mid u \in \mathcal{N}_d(v)\})$
      Compute preliminary gate $\hat{\mathbf{g}}_{v,d}^{(l)}$ (Eq. 5)
      Compute reset gate $\mathbf{r}_{v,d}^{(l)}$ and update $\tilde{\mathbf{g}}_{v,d}^{(l)}$ (Eq. 6)
      $\mathbf{h}_{v,d}^{(l)} = \tilde{\mathbf{g}}_{v,d}^{(l)} \circ \mathbf{h}_{v,d}^{(l-1)} + (1 - \tilde{\mathbf{g}}_{v,d}^{(l)}) \circ \mathbf{m}_{v,d}^{(l)}$
    end for
  end for
end for
Compute branch classification loss L_{branch} using $\mathbf{h}_{\leftarrow}^{(L)}$ and $\mathbf{h}_{\rightarrow}^{(L)}$
Detach directional embeddings before fusion: $\bar{\mathbf{h}}_{v,d}^{(L)} = \text{StopGrad}(\mathbf{h}_{v,d}^{(L)}), \forall d \in \{\leftarrow, \rightarrow\}, \forall v \in \mathcal{V}$
for $v \in \mathcal{V}$ **do**
  Incorporate structural embeddings: $\tilde{\mathbf{h}}_{v,d} = \bar{\mathbf{h}}_{v,d}^{(L)} + \beta \text{MLP}(\mathbf{a}_{v,d}), d \in \{\leftarrow, \rightarrow\}$
  Obtain fusion gates $q_{v,\leftarrow}, q_{v,\rightarrow}$ via Gaussian noise injected NTB block (Eq. 8)
  Compute fused representation: $\mathbf{h}_v^{\text{fused}} = \text{fuse}\left(q_{v,\leftarrow} \bar{\mathbf{h}}_{v,\leftarrow}^{(L)} + q_{v,\rightarrow} \bar{\mathbf{h}}_{v,\rightarrow}^{(L)}\right)$
end for
Compute classification loss L_{cls} using $\mathbf{h}^{\text{fused}}$
Compute importance loss $L_{imp} = \text{CV}\left(\sum_v q_{v,\leftarrow}, \sum_v q_{v,\rightarrow}\right)^2$
Calculate total loss $L_{total} = (1 - \lambda_2)(L_{cls} + \lambda_1 L_{imp}) + \lambda_2 L_{branch}$ and update model parameters via backpropagation

Table 1
Dataset properties and statistics. Edge Hom. is defined as the fraction of edges in the graph $\mathcal{G}$ that connect nodes with the same label.

	Cora-ML	Citeseer-Full	Chameleon	Squirrel	Roman-Empire
#Nodes	2995	4230	2277	5201	22,662
#Edges	8416	5358	36,101	217,073	44,363
#Features	2879	602	2325	2089	300
#Classes	7	6	5	5	18
Edge Hom.	0.792	0.949	0.235	0.223	0.044

graph constructed from a long Wikipedia article, where nodes are tokens linked by textual adjacency or dependency relations, with FastText-based features (Grave et al., 2018) and syntactic-role labels. Additional dataset details are provided in Table 1.

5.2. Experimental setup

The baseline models used for comparison fall into three categories: (1) Traditional undirected graph methods, such as GCN (Kipf & Welling, 2016), and GAT (Veličković et al., 2017); (2) State-of-the-art undirected graph methods for heterophilic graphs, including H2GCN (Zhu et al., 2020b), GPRGNN (Chien et al., 2020), LINKX (Lim et al., 2021), FSGNN (Maurya et al., 2021), ACM-GCN (Luan et al., 2022), GloGNN (Li et al., 2022), and G^2 (Gradient Gating) (Rusch et al., 2022); and (3) State-of-the-art directed graph methods, comprising DiGCN (Tong et al., 2020a), MagNet (Zhang et al., 2021), Dir-GNN (Rossi et al., 2024), and NDDGNN (Huang et al., 2024b).

For those baselines with node classification results reported in Huang et al. (2024b), we adopt the results in the paper. For DHGNN, we select the best hyperparameters from 100 runs using the Optuna hyperparameter optimization framework (Akiba et al., 2019). On other tasks, including link prediction, runtime evaluation, node classification at deeper layers, and the over-smoothing metrics and directional weights analysis, we subjected all baselines to the identical 100 Optuna runs used for DHGNN, thereby ensuring a fair comparison. The selected hyperparameters are detailed in Appendix C.

We conduct our experiments on an Intel Core i9-10940X CPU and 2 NVIDIA RTX A5000 GPUs.

Table 2
Node classification accuracy (%) across homophilic and heterophilic datasets. The best results are highlighted in **bold**, while the second-best results are underlined.

Datasets	Cora-ML	Citeseer-Full	Chameleon	Squirrel	Roman-Empire
MLP	77.48±1.23	80.01±1.23	46.36±2.57	34.14±1.94	65.76±0.42
GCN	52.37±1.77	54.65±1.22	64.82±2.24	53.43±2.01	73.69±0.74
GAT	54.12±1.56	55.15±1.31	45.56±3.16	39.14±2.88	71.16±0.63
H2GCN	62.86±1.45	68.34±1.36	59.39±0.98	37.90±0.02	60.11±0.52
GPR-GNN	68.88±1.66	70.12±1.24	62.85±2.90	54.35±0.87	64.85±0.27
ACM-GCN	69.96±1.53	73.61±1.32	74.76±2.20	67.40±2.21	69.66±0.62
GloGNN	73.78±1.69	76.13±1.14	57.88±1.76	71.21±1.84	59.63±0.69
LINKX	72.32±1.41	78.75±1.34	68.42±1.32	61.81±1.80	37.55±0.36
FSGNN	67.51±1.65	66.35±1.16	78.27±1.28	74.10±1.89	79.92±0.56
G^2	76.63±1.63	80.36±1.18	71.40±2.38	64.26±2.38	82.16±0.78
DiGCN	87.36±1.06	92.75±0.75	52.24±3.65	37.74±1.54	52.71±0.32
MagNet	85.26±1.05	93.38±0.89	58.32±2.87	39.01±1.93	88.07±0.27
Dir-GNN	84.45±1.69	92.79±0.59	79.71±1.26	75.31±1.92	91.23±0.32
NDDGNN	88.14±1.28	94.17±0.58	79.79±1.04	75.38±1.95	**91.76**±0.27
DHGNN	**88.39**±0.98	**94.60**±0.52	**80.11**±1.73	**76.84**±1.61	89.05±0.53

Table 3
Link prediction accuracy (%) on homophilic and heterophilic graphs. The best results are highlighted in **bold**, while the second-best results are underlined.

Datasets	Cora-ML	Citeseer-Full	Chameleon	Squirrel
GCN	76.24±4.25	71.16±4.41	86.03±1.53	90.64±0.49
GAT	64.58±12.28	67.28±2.90	85.12±1.57	90.37±0.45
MagNet	82.20±3.36	85.89±4.61	86.28±1.23	90.13±0.53
Dir-GNN	82.23±4.01	70.62±6.56	82.76±2.72	89.17±0.52
NDDGNN	83.04±9.56	87.93±3.92	90.30±1.82	92.40±1.01
DHGNN	**98.11**±1.84	**89.70**±6.76	**99.09**±2.02	**96.21**±3.42

5.3. Node classification

We evaluate our model on five datasets using the same data splits as in Huang et al. (2024b). We evaluate the performance by node classification accuracy with standard deviation in the semi-supervised setting. For Squirrel and Chameleon, we use 10 publicly-available splits (48%/32%/20% for training/validation/testing) provided by Pei et al. (2020). For the remaining datasets, we adopt the same splits as Rossi et al. (2024). The results, presented in Table 2, report the mean classification accuracy on the test nodes over 10 random splits. We also provide a sensitivity analysis of two hyperparameters for the node classification tasks. The corresponding results are presented in Appendix D.

Compared to methods designed for undirected heterophilic graphs, our proposed model consistently outperforms all undirected baselines. Specifically, it achieves average absolute improvements of 11.76%, 14.24%, 1.84%, 2.74% and 6.89% across the five datasets when compared to the strongest undirected method on each. It can be observed that MLP outperforms some of the GNN-based methods on directed datasets such as Cora-ML and Citeseer-Full. This suggests that these methods may struggle to effectively leverage structural information for node classification on directed graphs.

In addition, our model surpasses existing state-of-the-art methods tailored for directed graphs on four out of the five datasets. Take the heterophilic Squirrel dataset as an example. It exhibits the highest graph

Table 4
Ablation study for node classification. The best result (accuracy) is highlighted in **bold**. +**ResG.** denotes the resettable homophily-aware gating mechanism for message passing, +**SAF** denotes the Structure-Aware Fusion module, and +**NTB** denotes the Noise-Tolerant Block.

+ResG.	+SAF	$+L_{branch}$	$+L_{imp}$	+NTB	Cha.	Squ.	Rom.
-	–	–	–	–	79.21±1.03	74.96±1.95	43.35±0.63
✓	–	–	–	–	79.52±1.38	75.22±2.13	85.55±0.46
✓	✓	–	–	✓	79.98±1.68	76.38±1.57	86.50±0.36
✓	✓	✓	–	✓	79.54±1.17	76.25±1.84	88.06±0.44
✓	✓	✓	✓	–	79.82±1.20	75.98±1.99	88.37±0.58
✓	✓	✓	✓	✓	**80.11**±1.73	**76.84**±1.61	**89.05**±0.53

density and a relatively lower homophily ratio among the five datasets, indicating more complex inter-class linking patterns. The improvements on this dataset highlight our model's strong ability to leverage inter-class connections effectively. For the performance variation on Roman-Empire, we inspect the gating values on it and find a distinct pattern: its gating values are significantly larger and heavily skewed toward 1. This means that the gates in our DHGNN tend to saturate the representations and reject further neighborhood messages. We further check the characteristics of this dataset and find that the edge homophily ratio is significantly lower than that of the other datasets. Moreover, the dataset uses syntactic roles as node labels but with edges constructed by textual adjacency or dependency relations. Consequently, each node is linked pervasively to those with different syntactic roles but appearing in the same sentence, leading to an extraordinarily low homophily neighborhood. Our model, with the design goal of adaptively adjusting depth-wise information exchange based on homophily ratios, would encourage the gating values to rapidly increase and close early to block incoming messages when applied on such a dataset.

5.4. Link prediction

In addition to node classification, we assess the performance of the proposed DHGNN on link prediction tasks. In these tasks, the edges in directed graphs are treated as ordered pairs. Negative samples are generated using the negative_sampling method in PyG (Fey & Lenssen, 2019), following the same strategy as in Huang et al. (2024b). These negative samples exclude both the original edges and their reversed counterparts. The evaluation metric for link prediction is accuracy. This metric is not significantly affected by class imbalance since the number of negative samples matches that of positive edges. Although the traditional definition of heterophily based on node labels is not directly applicable in link prediction tasks, DHGNN can still capture the feature discrepancy between neighboring nodes. Results are reported in Table 3. Our model achieves the highest prediction accuracy across all datasets, with absolute improvements of 15.07%, 1.77%, 8.79%, and 3.81% over the best-performing baseline on each dataset, respectively. These results indicate that the proposed method effectively captures structural information from directed neighborhoods. This may be attributed to the fact that DHGNN maintains independent encoders for each direction, rather than performing fusion after every message passing layer in NDDGNN. This design enables each encoder to capture long-range dependencies along extended paths more effectively, thereby providing richer structural information for link prediction.

5.5. Ablation study

To evaluate the effectiveness of different components in our proposed framework, we conduct an ablation study focusing on five elements: the resettable homophily-aware gating mechanism (ResG.), the Structure-Aware Fusion module (SAF), the two auxiliary losses, i.e., L_{branch} and L_{imp}, and the noise-tolerant block (NTB). We ablate NTB with all other components on due to its interactions with other fusion components. Evaluating NTB while retaining all other fusion elements more accurately reflects its true contribution. When all components are disabled (the first row in Table 4), the model degenerates to a two-branch vanilla GCN that encodes each direction individually, and a linear layer followed by summation as the alternative fusion module. The results in Table 4 demonstrate that the homophily-aware gating mechanism significantly improves performance on heterophilic datasets. Additionally, the SAF module enhances the extraction of valuable information from the directional encoders, while NTB further improves the fusion quality between the two directions. The results also suggest that the combined use of the two auxiliary losses can lead to synergistic effects, further boosting classification accuracy.

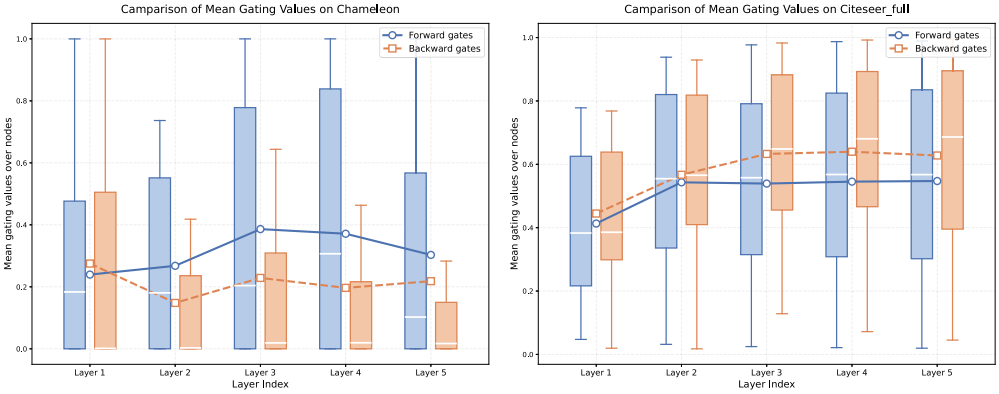
5.6. Visualization of gating values in message passing

To better understand the behavior of our homophily-aware gated message passing mechanism, we visualize the gating values on two representative datasets: the heterophilic Chameleon and the homophilic Citeseer-Full, as shown in Fig. 4. A clear gap in gating values between the two directions is observed on Chameleon, highlighting a strong directional asymmetry. While a similar trend is present on Citeseer-Full, the gap is notably smaller, suggesting that the gating mechanism adaptively responds to the directional differences in homophily on different types of graphs.

Additionally, we observe a decrease in gating values of the backward direction from layer 1 to layer 2 on the Chameleon dataset. This aligns with the higher homophily ratio at the second hop, as illustrated in Fig. 1a. The larger variance in gating values, as indicated by the taller boxplots, may be attributed to the increased variability in neighborhood homophily at greater hop distances. These findings indicate that the proposed gating mechanism not only distinguishes between directional neighborhoods but also dynamically adapts to the non-monotonic variations in homophily across layers. Similar gating behavior is observed on other datasets, as visualized in Appendix D.

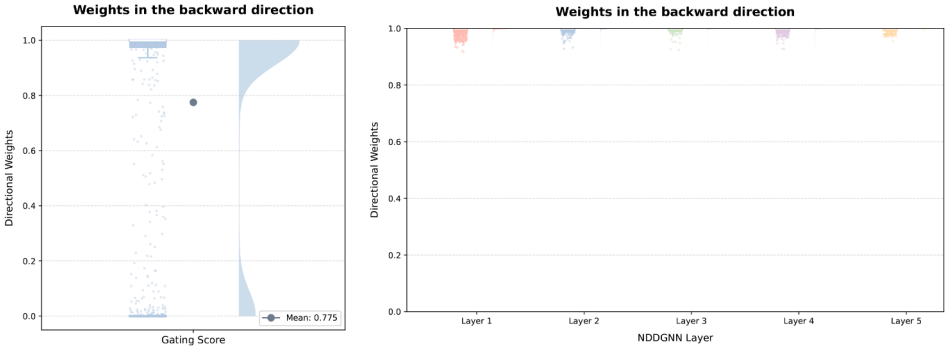
5.7. Analysis of the fusion module

DHGNN allows a more balanced choice between the forward and backward directions through our design of structure-aware noise-tolerant fusion. We visualize the weights of the backward direction of both DHGNN and NDDGNN on Chameleon in Fig. 5. Note that NDDGNN performs fusion at each layer and thus we show its layer-wise weights. We can see from Fig. 5b that NDDGNN weights concentrate overwhelmingly above 0.98 across all five layers. As the weights in the forward direction are the strict complement of the weights visualized here, forces the model to rely almost entirely on a single direction, failing to capture vital structural information from the opposite direction. In contrast, our DHGNN exhibits a more balanced allocation in message passing. The distribution of gating values shown in Fig. 5a confirms that our design prevents fusion from skewing entirely in one direction and allows a proportion of nodes to favor embeddings from the opposite direction. The



(a) Forward/backward gating values on Chameleon. (b) Forward/backward gating values on Citeseer-Full.

Fig. 4. The gating values of the first five layers on heterophilic and homophilic datasets. The dots on line charts are the mean value over both nodes and chunks, while the box-plots present the distribution of mean gating values over chunks.



(a) DHGNN. (b) NDDGNN.

Fig. 5. The raincloud plots of the weights in the backward direction on Chameleon.

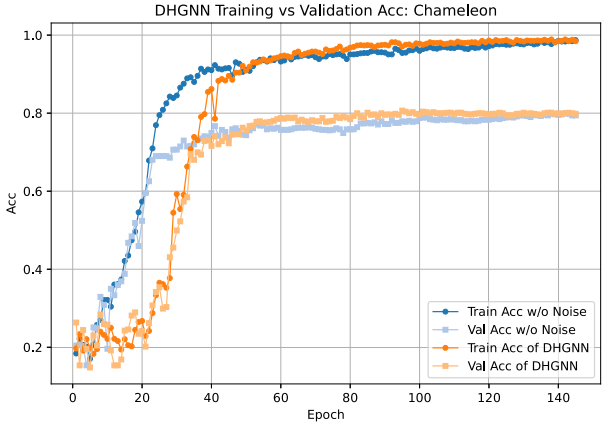


Fig. 6. Convergence curves of DHGNN with and without NTB on Chameleon.

mean gating value also indicates that our forward and backward encoders adapt well across layers without unidirectional collapse.

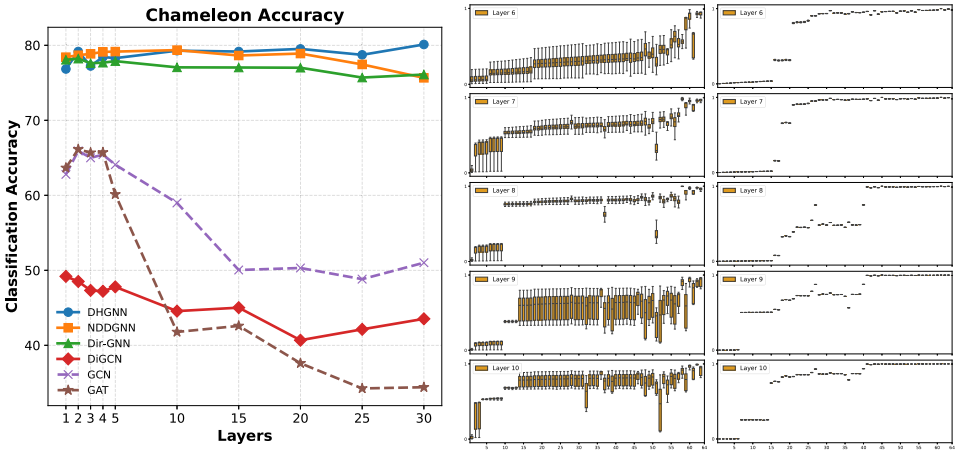
To further demonstrate how NTB in the fusion affects training, we plot the convergence curves on Chameleon in Fig. 6. We observe a longer exploration at the early training of DHGNN with NTB, which ultimately drives the model to converge at a higher accuracy. Specifically, the version without NTB exhibits an immediate but jittery ascent starting from epoch 0, while the one with NTB undergoes an exploration phase with accuracy staying near-random for the first 20 epochs (this is a 5-class classification task and thus the random accuracy is 0.2). This exploration leads to a higher validation accuracy when compared to the one without NTB.

5.8. *Alleviating over-smoothing in DHGNN*

As shown in Fig. 7, DHGNN effectively mitigates the over-smoothing problem. Node classification accuracy does not decrease, and even slightly improves, as the model goes deeper. The change in accuracy is obviously smaller than that observed in traditional GNNs, and remains lower than other baselines. Analysis of the gating vectors at deeper layers (Fig. 7b) shows that while some gates saturate and refuse neighborhood messages, others remain open with lower values. This suggests that the model continues to capture informative signals from distant nodes, even at greater depths. We visualize the results on additional datasets in Appendix F, which exhibit similar patterns to those in Fig. 7a, further suggesting that DHGNN effectively alleviates over-smoothing on heterophilic graphs.

Apart from the node classification accuracy, we evaluate the level of over-smoothing with some additional metrics: Normalized Dirichlet Energy (NDE) (Cai & Wang, 2020), Mean Average Distance (MAD) (Chen et al., 2020), Stable Rank (StabRank) (Arora et al., 2019), and Numerical Rank (NumRank) (Rudelson & Vershynin, 2007). The definitions of these metrics can be found in Appendix G.

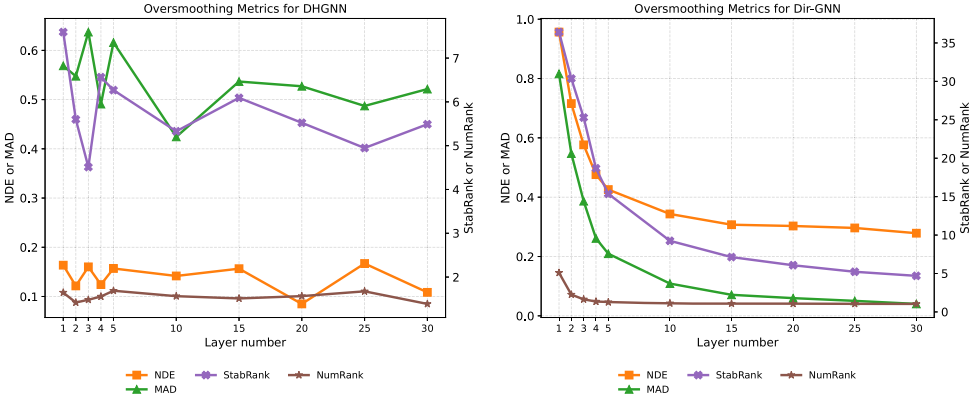
We visualized these metrics for our DHGNN and the baseline DirGNN, on the Chameleon dataset with the number of layers tuned from 1 to 30. A decreasing trend of each metric as the number of layers increases indicates over-smoothing. We plot the results separately for each model in Fig. 8, as it is invalid to compare the magnitudes of these metrics across different models (Zhang et al., 2026). The secondary y-axis on the right is for the two rank-based metrics.



(a) Node classification accuracy on Chameleon at deeper layers, better visualized in color.

(b) Gating values on Chameleon at deeper layers.

Fig. 7. The node classification accuracy and behavior of gating values at deeper layers.



(a) DHGNN.

(b) Dir-GNN.

Fig. 8. Over-smoothing metrics on Chameleon, better visualized in color.

Table 5
Comparison of runtime per-epoch (in seconds).

	Cora.	Citeseer.	Chameleon	Squirrel	Roman.
NDDGNN	0.0634	0.0529	0.0643	0.1267	0.1430
DHGNN	0.0455	0.1191	0.1026	0.1165	0.2090

It can be observed that while DHGNN's curves fluctuate, the metric values remain comparable between extremely deep and shallow models. In contrast, Dir-GNN exhibits a clear and significant decreasing trend at all metrics. These results confirm that our DHGNN is able to effectively alleviate over-smoothing.

5.9. Runtime evaluation

To examine the efficiency of DHGNN, we compare it with the GCN-like baseline NDDGNN (Huang et al., 2024b). Both models use 4 layers and the same hidden dimension. The reported time in Table 5 is the average per-epoch cost, including training and validation, measured in seconds.

The training time of DHGNN is on the same order of magnitude as NDDGNN. Although DHGNN is slightly slower on some datasets, it remains efficient and even faster on the densest graph with the highest edge count, Squirrel, indicating that the gating mechanism introduces only moderate computational overhead. Though the improvements of DHGNN over NDDGNN on node classification are not pronounced, its improvements on link prediction are very significant (up to 15.07%). Given that the time difference is at an acceptable level (up to ~2 times), DHGNN presents a worthwhile tradeoff between computational overhead and performance on downstream tasks.

6. Conclusion

We introduced Directed Homophily-aware Graph Neural Network (DHGNN), a novel framework that adaptively integrates neighborhood information by accounting for both homophily gaps between opposite directions and homophily fluctuations across layers. DHGNN features a homophily-aware gating mechanism with a resettable update strategy and a structure-aware, noise-tolerant fusion module that selectively emphasizes informative directional representations. Comprehensive experiments on both homophilic and heterophilic datasets show that DHGNN can outperform state-of-the-art methods on node classification and link prediction tasks. Gating analyses further confirm its ability to adaptively respond to complex homophily patterns. Our method not only advances effective learning on directed graphs but also introduces a robust mechanism for extracting informative signals from obscure or noisy neighborhoods.

While DHGNN has proven effective, its scalability on extremely large graphs and generalization to broader domains beyond citation and web graphs remain promising directions for future exploration. We aim to

further enhance the model's efficiency and extend its applicability across diverse graph domains, paving the way for broader adoption in real-world scenarios.

CRediT authorship contribution statement

Aihu Zhang: Writing – original draft, Visualization, Methodology, Formal analysis; **Jiaxing Xu:** Writing – review & editing, Methodology; **Mengcheng Lan:** Writing – review & editing, Methodology; **Shili Xiang:** Writing – review & editing, Funding acquisition; **Yiping Ke:** Writing – review & editing, Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This research/project is supported by the Ministry of Education, Singapore under its Academic Research Fund Tier 1 (RG16/24). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of the Ministry of Education, Singapore.

Appendix A. Analysis of homophily ratio with SBM

To illustrate that homophily may increase with hop distance, we consider a directed two-block SBM with classes A, B (balanced with n nodes in each class for simplicity). Each directed edge (i, j) appears independently with:

$$\mathbb{P}[i \to j] = \begin{cases} p_{\text{in}}, & \text{if class}(i) = \text{class}(j), \\ p_{\text{out}}, & \text{otherwise.} \end{cases}$$

The expected one-hop homophily is

$$h_1(u) = \frac{(n-1)p_{\text{in}}}{(n-1)p_{\text{in}} + np_{\text{out}}}.$$

When $p_{\text{out}} > p_{\text{in}}$, this value is small, indicating a heterophilous 1-hop neighborhood.

To compute the 2-hop homophily, we consider all 2-hop paths $u \to w \to v$, where w is a 1-hop neighbor and v is a 2-hop neighbor of u. Let $h_2(u)$ denote the probability that a randomly selected 2-hop neighbor v shares the same class as u. This can be decomposed based on the class of the intermediate node w. The probability that a 1-hop neighbor w belongs to class A is:

$$\mathbb{P}(w \in A \mid u \in A) = \frac{(n-1)p_{\text{in}}}{(n-1)p_{\text{in}} + np_{\text{out}}},$$

while $w \in B$ with probability:

$$\mathbb{P}(w \in B \mid u \in A) = \frac{np_{\text{out}}}{(n-1)p_{\text{in}} + np_{\text{out}}}.$$

Conditional on w's class, the probability that the 2-hop neighbor $v \in A$ is given by:

$$\mathbb{P}(v \in A \mid w \in A) = \frac{(n-1)p_{\text{in}}}{(n-1)p_{\text{in}} + np_{\text{out}}},$$
$$\mathbb{P}(v \in A \mid w \in B) = \frac{np_{\text{out}}}{(n-1)p_{\text{in}} + np_{\text{out}}}.$$

Thus, the 2-hop homophily is:

$$h_2(u) = \left(\frac{(n-1)p_{\text{in}}}{(n-1)p_{\text{in}} + np_{\text{out}}}\right)^2 + \left(\frac{np_{\text{out}}}{(n-1)p_{\text{in}} + np_{\text{out}}}\right)^2.$$

When $p_{\text{out}} > p_{\text{in}}$, for instance, if we fix the class size $n = 500$, the intra-class and inter-class connection probabilities $p_{\text{in}} = 0.2$ and $p_{\text{out}} = 0.8$, the two homophily ratios $h_1(u)$, and $h_2(u)$ are 0.1999 and 0.6804 respectively. Under this setting, $h_2(u)$ exceeds $h_1(u)$, indicating that homophily may increase at the 2-hop level. This illustrates that homophily is not necessarily a monotonically decreasing function of hop distance and suggests that the design of GNNs for heterophilic graphs should consider the fluctuations of homophily ratio in the aggregation of multiple layers.

Appendix B. Monotonicity analysis of the homophily-aware resettable gate

Consider the same directed SBM model, but now with class sizes n_A and n_B. To keep algebra transparent, we adopt scalar embeddings $\mathbf{h}_v^{(l)} \in \mathbb{R}$ for each node. Initially $\mathbf{h}_v^{(0)} = +1$ if $v \in A$, and $\mathbf{h}_v^{(0)} = -1$ if $v \in B$. Under such settings, the expected one-hop homophily is

$$h_1 = \frac{(n_A - 1)p_{\text{in}}}{(n_A - 1)p_{\text{in}} + n_B p_{\text{out}}}.$$

Lemma 1 (Expected neighbor average)**.** *For $v \in A$ under mean-field approximation, the l-hop message can be expressed as:*

$$\mathbb{E}[\mathbf{m}_v^{(l)}] = (+1) \cdot h_l + (-1) \cdot (1 - h_l) = 2h_l - 1,$$

where h_l is the l-hop homophily defined by the proportion of nodes in class A in the l-hop neighbors of the central node.

Suppose the similarity between the center node v and its l-hop neighborhood is defined by inner product between the node representation and the current message:

$$\mathbb{E}\left[s_v^{(l)}\right] = \mathbb{E}\left[\mathbf{x}_v \mathbf{m}_v^{(l)}\right] = 2h_l - 1.$$

The same result can be derived for nodes in class B.

Assume that the gate proposal at layer l is an affine decreasing function of the node-message similarity, then it can also be expressed as an affine decreasing function of the expected homophily ratio h_l:

$$\hat{\mathbf{g}}_v^{(l)} = \alpha_1 - \alpha_2 s_v^{(l)} = \alpha_1 + \alpha_2 - 2\alpha_2 h_l \qquad \alpha_2 > 0,$$

where α_1, α_2 are weights of the affine function.

Theorem 1 (Gate Non-Monotonicity)**.** *In DHGNN, there exist graph structures and parameter configurations where the gate sequence $\{\tilde{\mathbf{g}}_v^{(l)}\}_{l=0}^{L}$ is non-monotonic.*

Proof. Consider a directed SBM with parameters: $n = 500$, $p_{\text{in}} = 0.2$, $p_{\text{out}} = 0.8$, yielding homophily ratios $h_1 \approx 0.20$, $h_2 \approx 0.68$. Without loss of generality, we assume $\alpha_1 = \alpha_2 = 1/2$ and $\hat{\mathbf{g}}_v^{(l)} = \mathbf{r}_v^{(l)} = 1 - h_l$ in the following analysis:

Layer 1:

- High heterophily ($h_1 \approx 0.20$) $\Rightarrow$ Small $s_v^{(1)}$, $\hat{\mathbf{g}}_v^{(1)} \approx 0.80$
- With $\tilde{\mathbf{g}}_v^{(0)} = 0$, we have $\hat{\mathbf{g}}_v^{(1)} > \tilde{\mathbf{g}}_v^{(0)}$
- Reset gate $\mathbf{r}_v^{(1)} \approx 0.80 \Rightarrow \tilde{\mathbf{g}}_v^{(1)} \approx 0.80 \times 0.80 + 0.20 \times 0 = 0.64$

Layer 2:

- Higher homophily ($h_2 \approx 0.68$) $\Rightarrow$ Large $s_v^{(2)}$, $\hat{\mathbf{g}}_v^{(2)} \approx 0.32$
- With $\tilde{\mathbf{g}}_v^{(1)} \approx 0.64$, we have $\hat{\mathbf{g}}_v^{(2)} < \tilde{\mathbf{g}}_v^{(1)}$
- Reset gate $\mathbf{r}_v^{(2)} \approx 0.32 \Rightarrow \tilde{\mathbf{g}}_v^{(2)} \approx 0.32 \times 0.32 + 0.68 \times 0.64 = 0.5376$

This demonstrates:

- $\tilde{\mathbf{g}}_v^{(1)} > \tilde{\mathbf{g}}_v^{(0)}$ (increase from layer 0 to 1)
- $\tilde{\mathbf{g}}_v^{(2)} < \tilde{\mathbf{g}}_v^{(1)}$ (decrease from layer 1 to 2)

Hence, the gate sequence is non-monotonic. □

Reset gate dynamics. The reset gate $\mathbf{r}_v^{(l)}$ controls the interpolation between preservation and adaptation:

$$\tilde{\mathbf{g}}_v^{(l)} = (1 - \mathbf{r}_v^{(l)}) \circ \tilde{\mathbf{g}}_v^{(l-1)} + \mathbf{r}_v^{(l)} \circ \hat{\mathbf{g}}_v^{(l)}$$

The gate difference between consecutive layers is:

$$\Delta\tilde{\mathbf{g}}_v^{(l)} = \tilde{\mathbf{g}}_v^{(l)} - \tilde{\mathbf{g}}_v^{(l-1)} = \mathbf{r}_v^{(l)} \circ \left(\hat{\mathbf{g}}_v^{(l)} - \tilde{\mathbf{g}}_v^{(l-1)}\right)$$

Since $\mathbf{r}_v^{(l)} \geq 0$ element-wise, the sign of $\Delta\tilde{\mathbf{g}}_v^{(l)}$ is determined by $\text{sign}(\hat{\mathbf{g}}_v^{(l)} - \tilde{\mathbf{g}}_v^{(l-1)})$.

Lemma 2 (Reset Gate Bounds)**.** *The reset gate modulates but cannot reverse the direction of gate change:*

- *If* $\hat{\mathbf{g}}_v^{(l)} > \tilde{\mathbf{g}}_v^{(l-1)}$, *then* $\tilde{\mathbf{g}}_v^{(l)} \geq \tilde{\mathbf{g}}_v^{(l-1)}$
- *If* $\hat{\mathbf{g}}_v^{(l)} < \tilde{\mathbf{g}}_v^{(l-1)}$, *then* $\tilde{\mathbf{g}}_v^{(l)} \leq \tilde{\mathbf{g}}_v^{(l-1)}$
- *Equality occurs only when* $\mathbf{r}_v^{(l)} = 0$

This analysis demonstrates that DHGNN's resettable gating mechanism enables non-monotonic gate behavior in response to fluctuating homophily patterns across layers. The reset gate $\mathbf{r}_v^{(l)}$ provides adaptive control over this behavior, allowing the model to effectively handle complex directed graphs with varying homophily structures.

Appendix C. Hyperparameter settings

The detailed settings of the hyperparameters in both the encoders and the fusion mechanism of the proposed DHGNN are presented in Table C.1.

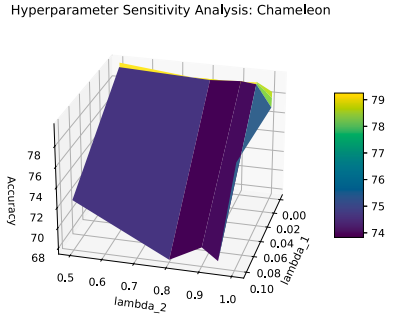
Table C.1
Hyperparameter settings for different datasets.

Hyperparameter	Cor.	Cit.	Cha.	Squ.	Rom.
lr	0.001	0.005	0.05	0.01	0.005
wd	1e-6	0.0005	0.0001	0.0001	5e-6
GNN layers	4	14	12	5	6
hidden dim	256	256	256	256	256
gate_mlp layers	2	2	3	2	2
adj_mlp layers	2	2	4	2	4
input_fc dropout	0.5	0.5	0.5	0.5	0.3
dropout rate	0.2	0.0	0.3	0.3	0.1
adj_coef β	0	0.4	0.5	0.5	0
imp_coef λ_1	1e-3	1e-7	1e-5	1e-7	1e-3
branch_coef λ_2	0.9	0.95	0.95	0.8	0.95
chunk size	64	64	64	64	64

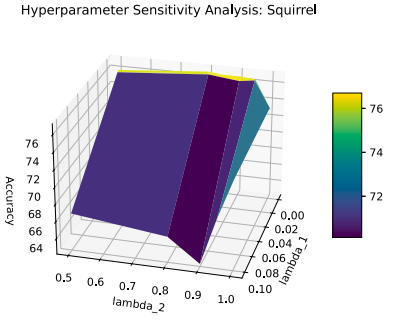
Appendix D. Hyperparameter sensitivity analysis

To further understand the robustness and sensitivity of our model, we provide a hyperparameter sensitivity analysis in this section. Fig. D.1 illustrates the performance variations under different settings of λ_1 and λ_2.

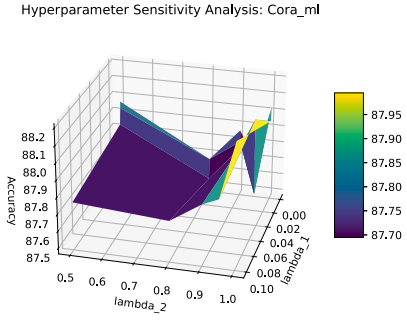
On the Chameleon and Squirrel datasets, the best performance is achieved when λ_1 is close to 0, indicating that the model favors embed-



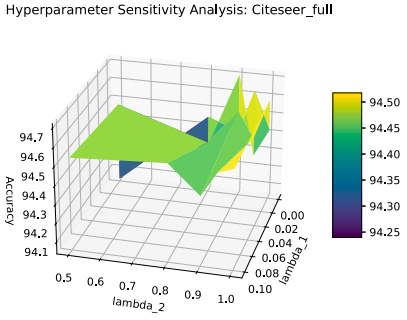
(a) Hyperparameter sensitivity analysis on Chameleon.



(b) Hyperparameter sensitivity analysis on Squirrel.



(c) Hyperparameter sensitivity analysis on Cora_ML.



(d) Hyperparameter sensitivity analysis on Citeseer-Full.

Fig. D.1. Hyperparameter sensitivity analysis on various datasets.

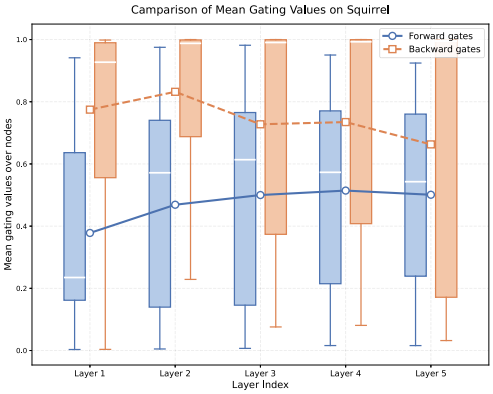
dings from a single direction on these heterophilic graphs and tends to assign a larger weight during fusion. As a result, the constraint enforcing balanced decision scores can be relaxed in this case.

For the hyperparameter λ_2, the best node classification performance is typically achieved when its value is close to 1, suggesting that the final representation is more reliably derived from the higher-quality single-directional embeddings. This also implies that the post-fusion classification loss plays a relatively minor role in the overall optimization.

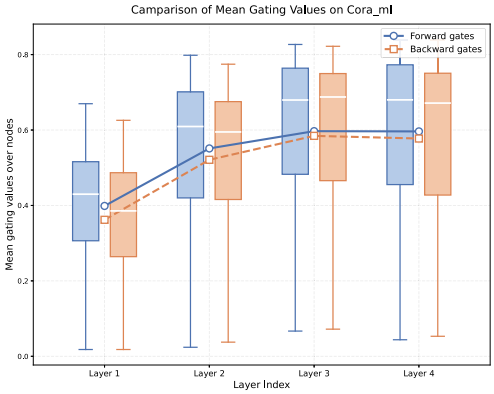
From the visualization, we may conclude that, although the best performance is sometimes achieved when $\lambda_1 = 0$, indicating the exclusion of the corresponding importance loss, both the importance loss and the branch classification loss still contribute meaningfully across different datasets. The hyperparameter analysis further confirms the effectiveness of both auxiliary losses.

Appendix E. Visualization of the gating vectors on the other datasets

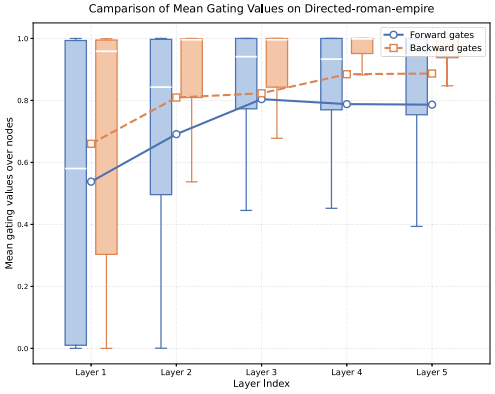
In addition to the visualization in Figs. 4, E.1 presents gating values from other datasets, further illustrating the directional variance and the changes across different layers.



(a) Forward/backward gating values on Squirrel.

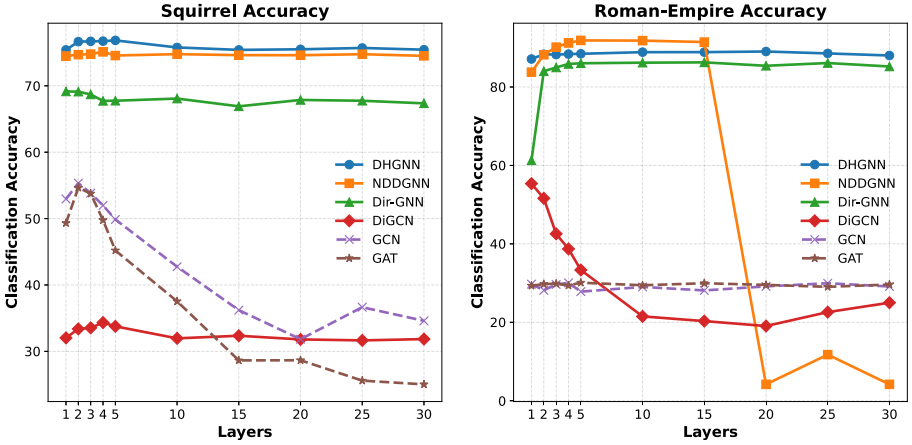


(b) Forward/backward gating values on Cora-ML.



(c) Forward/backward gating values on Roman-Empire.

Fig. E.1. The gating values of the first five layers on the other datasets. The dots on line charts are the mean value over both nodes and chunks, while the box-plots present the distribution of mean gating values over chunks.



(a) Node classification accuracy on Squirrel at deeper layers, better visualized in color.

(b) Node classification accuracy on Roman-Empire at deeper layers, better visualized in color.

Fig. E.2. The node classification accuracy on other heterophilic datasets at deeper layers.

On both the Squirrel and Roman-Empire datasets, we observe some decreasing cases in the gating values, suggesting the presence of meaningful information from more distant hops.

In contrast, for datasets where embeddings from the two directions are highly similar, such as Cora-ML, the gating mechanism adapts accordingly, exhibiting consistent behavior across layers and learning from neighborhood information in a balanced manner.

Appendix F. Node classification results on other datasets at deeper layers

Beyond the visualization in Figs. 7, E.2 shows the accuracy curves for node classification on other heterophilic datasets. From these two curves, we observe that the node classification performance of both DHGNN and Dir-GNN remains relatively stable as the number of layers increases, with DHGNN consistently achieving higher accuracy. NDDGNN also demonstrates stable performance on the Squirrel dataset, but its accuracy drops sharply beyond 20 layers on Roman-Empire, possibly due to instability in directional fusion at each layer, leading to node embeddings that converge to indistinguishable representations. These results indicate that DHGNN effectively mitigates the over-smoothing problem on heterophilic graphs.

Appendix G. Definitions of the over-smoothing metrics

Normalized Dirichlet Energy (NDE). Cai and Wang (2020) The Dirichlet Energy measures the smoothness of the node features $\mathbf{X}$ over the graph structure, and the normalized term is defined as:

$$\mathrm{NDE}(\mathbf{X}) = \frac{E_{\mathrm{Dir}}(\mathbf{X})}{\|\mathbf{X}\|_F^2},$$

and

$$E_{\mathrm{Dir}}(\mathbf{X}) = \sum_{(i,j)\in\mathcal{E}} \left\| \frac{\mathbf{X}_{i,:}}{\mathbf{u}_i} - \frac{\mathbf{X}_{j,:}}{\mathbf{u}_j} \right\|_2^2,$$

where $\mathcal{E}$ is the set of all edges, $\mathbf{u}_i$ is the ith entry of the dominant eigenvector of the message-passing matrix. The message-passing matrix represents how node features are weighted and aggregated from their neighbors during a network layer's forward pass. In standard models like GCNs, it is typically represented by the symmetrically normalized adjacency matrix ($\tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}$), which strictly enforces the graph's structural topology during information flow.

Mean Average Distance (MAD). Chen et al. (2020) It calculates the average cosine distance across all edges in the graph:

$$\mathrm{MAD}(\mathbf{X}) = \frac{1}{|\mathcal{E}|} \sum_{(i,j)\in\mathcal{E}} \left(1 - \frac{\mathbf{X}_{i,:}^\top \mathbf{X}_{j,:}}{\|\mathbf{X}_{i,:}\|_2 \|\mathbf{X}_{j,:}\|_2} \right).$$

Stable Rank (StabRank). Arora et al. (2019) It is defined as:

$$\mathrm{StabRank}(\mathbf{X}) = \frac{\|\mathbf{X}\|_*^2}{\|\mathbf{X}\|_F^2},$$

where $\|\mathbf{X}\|_* = \sum_i \sigma_i$ is the nuclear norm, and σ_i denotes the ith singular value of the feature matrix $\mathbf{X}$.

Numerical Rank (NumRank). Rudelson and Vershynin (2007) It is defined as:

$$\mathrm{NumRank}(\mathbf{X}) = \frac{\|\mathbf{X}\|_F^2}{\|\mathbf{X}\|_2^2}.$$

References

Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *The 25th ACM SIGKDD International conference on knowledge discovery & data mining* (pp. 2623–2631).

Arora, S., Cohen, N., Hu, W., & Luo, Y. (2019). Implicit regularization in deep matrix factorization. In *Advances in neural information processing systems*, *32*.

Bojchevski, A., & Günnemann, S. (2017). Deep Gaussian embedding of graphs: Unsupervised inductive learning via ranking. arXiv:1707.03815.

Cai, C., & Wang, Y. (2020). A note on over-smoothing for graph neural networks. arXiv:2006.13318.

Chen, D., Lin, Y., Li, W., Li, P., Zhou, J., & Sun, X. (2020). Measuring and relieving the over-smoothing problem for graph neural networks from the topological view. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 3438–3445). (*vol. 34*).

Chen, J., Lei, R., & Wei, Z. (2024). PolyGCL: Graph contrastive learning via learnable spectral polynomial filters. In *The twelfth international conference on learning representations*.

Chien, E., Peng, J., Li, P., & Milenkovic, O. (2020). Adaptive universal generalized pagerank graph neural network. arXiv:2006.07988.

Fey, M., & Lenssen, J. E. (2019). Fast graph representation learning with PyTorch geometric. In *ICLR workshop on representation learning on graphs and manifolds*.

Gai, Y., & Liu, J. (2025). Interpretable unsupervised neural network structure for data clustering via differentiable reconstruction of ONMF and sparse autoencoder. *Neural Networks*, *188*, 107504.

Geisler, S., Li, Y., Mankowitz, D. J., Cemgil, A. T., Günnemann, S., & Paduraru, C. (2023). Transformers meet directed graphs. In *International conference on machine learning* (pp. 11144–11172). PMLR.

Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., & Dahl, G. E. (2017). Neural message passing for quantum chemistry. In *International conference on machine learning* (pp. 1263–1272). PMLR.

Grave, E., Bojanowski, P., Gupta, P., Joulin, A., & Mikolov, T. (2018). Learning word vectors for 157 languages. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018)* (pp. 3483–3487). Miyazaki, Japan, European Language Resources Association (ELRA).

Gu, M., Zheng, Z., Zhou, S., Liu, M., Chen, J., Qiao, T., Li, L., & Bu, J. (2024). Making classic GNNs strong baselines across varying homophily: A smoothness-generalization perspective. arXiv:2412.09805.

Guo, X., Zhang, P., Cai, Q., & Zhang, Y. (2025). Learning to solve combinatorial optimization problems with heterophily. *Neural Networks*, 107554.

Hamilton, W., Ying, Z., & Leskovec, J. (2017). Inductive representation learning on large graphs. In *Advances in neural information processing systems*, *30*.

He, D., Liang, C., Liu, H., Wen, M., Jiao, P., & Feng, Z. (2022). Block modeling-guided graph convolutional neural networks. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 4022–4029). (*vol. 36*).

Huang, C., Wang, Y., Jiang, Y., Li, M., Huang, X., Wang, S., Pan, S., & Zhou, C. (2024a). Flow2GNN: Flexible two-way flow message passing for enhancing gnns beyond homophily. *IEEE Transactions on Cybernetics*, *54* (11), 6607–6618.

Huang, J., Mo, Y., Hu, P., Shi, X., Yuan, S., Zhang, Z., & Zhu, X. (2024b). Exploring the role of node diversity in directed graph representation learning. In *Proceedings of the thirty-third international joint conference on artificial intelligence (IJCAI-24)* (pp. 2072–2080). York, United Kingdom: International Joint Conferences on Artificial Intelligence Organization.

Huang, Y., Wang, H., & Li, P. (2024c). What are good positional encodings for directed graphs? arXiv:2407.20912.

Huang, Y., Zhao, J., He, D., Wang, X., Li, Y., Huang, Y., Jin, D., & Feng, Z. (2025). One prompt fits all: Universal graph adaptation for pretrained models. In *The thirty-ninth annual conference on neural information processing systems*.

Khemani, B., Patil, S., Kotecha, K., & Tanwar, S. (2024). A review of graph neural networks: Concepts, architectures, techniques, challenges, datasets, applications, and future directions. *Journal of Big Data*, *11* (1), 18.

Kipf, T. N., & Welling, M. (2016). Semi-supervised classification with graph convolutional networks. arXiv:1609.02907.

Li, M., Gu, Y., Wang, Y., Fang, Y., Bai, L., Zhuang, X., & Lio, P. (2025). When hypergraph meets heterophily: New benchmark datasets and baseline. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 18377–18384). (*vol. 39*).

Li, Q., Han, Z., & Wu, X.-M. (2018). Deeper insights into graph convolutional networks for semi-supervised learning. In *Proceedings of the AAAI conference on artificial intelligence*. (*vol. 32*).

Li, X., Zhu, R., Cheng, Y., Shan, C., Luo, S., Li, D., & Qian, W. (2022). Finding global homophily in graph neural networks when meeting heterophily. In *International conference on machine learning* (pp. 13242–13256). PMLR.

Lim, D., Hohne, F., Li, X., Huang, S. L., Gupta, V., Bhalerao, O., & Lim, S. N. (2021). Large scale learning on non-homophilous graphs: New benchmarks and strong simple methods. *Advances in Neural Information Processing Systems*, *34*, 20887–20902.

Luan, S., Hua, C., Lu, Q., Zhu, J., Zhao, M., Zhang, S., Chang, X.-W., & Precup, D. (2022). Revisiting heterophily for graph neural networks. *Advances in Neural Information Processing Systems*, *35*, 1362–1375.

Maurya, S. K., Liu, X., & Murata, T. (2021). Improving graph neural networks with simple architecture design. arXiv:2105.07634.

Pei, H., Wei, B., Chang, K. C.-C., Lei, Y., & Yang, B. (2020). Geom-GCN: Geometric graph convolutional networks. arXiv:2002.05287.

Platonov, O., Kuznedelev, D., Diskin, M., Babenko, A., & Prokhorenkova, L. (2023). A critical look at the evaluation of GNNs under heterophily: Are we really making progress? arXiv:2302.11640.

Rossi, E., Charpentier, B., Di Giovanni, F., Frasca, F., Günnemann, S., & Bronstein, M. M. (2024). Edge directionality improves learning on heterophilic graphs. In *Learning on graphs conference* (pp. 25–1). PMLR.

Rudelson, M., & Vershynin, R. (2007). Sampling from large matrices: An approach through geometric functional analysis. *Journal of the ACM (JACM)*, *54* (4), 21–es.

Rusch, T. K., Chamberlain, B. P., Mahoney, M. W., Bronstein, M. M., & Mishra, S. (2022). Gradient gating for deep multi-rate learning on graphs. arXiv:2210.00513.

Scarselli, F., Gori, M., Tsoi, A. C., Hagenbuchner, M., & Monfardini, G. (2009). The graph neural network model. *IEEE Transactions on Neural Networks*, 61–80. https://doi.org/10.1109/tnn.2008.2005605

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., & Dean, J. (2017). Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. arXiv:1701.06538.

Shi, Z., & Liu, J. (2023). Noise-tolerant clustering via joint doubly stochastic matrix regularization and dual sparse coding. *Expert Systems with Applications*, *222*, 119814.

Song, Y., Zhou, C., Wang, X., & Lin, Z. (2023). Ordered GNN: Ordering message passing to deal with heterophily and over-smoothing. arXiv:2302.01524.

Suresh, S., Budde, V., Neville, J., Li, P., & Ma, J. (2021). Breaking the limit of graph neural networks by improving the assortativity of graphs with local mixing patterns. In *Proceedings of the 27th ACM SIGKDD Conference on knowledge discovery & data mining* (pp. 1541–1551).

Tang, B., Chen, X., Wang, S., Xuan, Y., & Zhao, Z. (2023). Generalized heterophily graph data augmentation for node classification. *Neural Networks*, *168*, 339–349.

Tong, Z., Liang, Y., Sun, C., Li, X., Rosenblum, D., & Lim, A. (2020a). Digraph inception convolutional networks. *Advances in Neural Information Processing Systems*, *33*, 17907–17918.

Tong, Z., Liang, Y., Sun, C., Rosenblum, D. S., & Lim, A. (2020b). Directed graph convolutional network. arXiv:2004.13970.

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., & Bengio, Y. (2017). Graph attention networks. arXiv:1710.10903.

Wang, H., Jiang, Z., You, Y., Han, Y., Liu, G., Srinivasa, J., Kompella, R. R., & Wang, Z. (2023). Graph mixture of experts: Learning on large-scale graphs with explicit diversity modeling. *Advances in Neural Information Processing Systems*, *36*, 50825–50837.

Wang, X., & Liu, J. (2025). Multi-spectral attention and graph smoothness enhancement for generalized node classification. *Neural Networks*, 108287.

Wang, Y., Huang, C., Li, M., Cai, T., Zheng, Z., & Huang, X. (2025). All roads lead to rome: Exploring edge distribution shifts for heterophilic graph learning. In *Proceedings of the thirty-fourth international joint conference on artificial intelligence* (pp. 6487–6495).

Xie, X., Chen, W., & Kang, Z. (2025). Robust graph structure learning under heterophily. *Neural Networks*, *185*, 107206.

Xu, K., Hu, W., Leskovec, J., & Jegelka, S. (2018). How powerful are graph neural networks? arXiv:1810.00826.

Yan, Y., Hashemi, M., Swersky, K., Yang, Y., & Koutra, D. (2022). Two sides of the same coin: Heterophily and oversmoothing in graph convolutional neural networks. In *2022 IEEE International conference on data mining (ICDM)* (pp. 1287–1292). IEEE.

Zhang, K., Deidda, P., Higham, D. J., & Tudisco, F. (2026). Are we measuring oversmoothing in graph neural networks correctly? In *International conference on learning representations (ICLR 2026)*.

Zhang, X., He, Y., Brugnone, N., Perlmutter, M., & Hirn, M. (2021). MagNet: A neural network for directed graphs. *Advances in Neural Information Processing Systems*, *34*, 27003–27015.

Zhu, J., Yan, Y., Zhao, L., Heimann, M., Akoglu, L., & Koutra, D. (2020a). Beyond homophily in graph neural networks: Current limitations and effective designs. *Advances in Neural Information Processing Systems*, *33*, 7793–7804.

Zhu, J., Yan, Y., Zhao, L., Heimann, M., Akoglu, L., & Koutra, D. (2020b). Beyond homophily in graph neural networks: Current limitations and effective designs. *Advances in Neural Information Processing Systems*, *33*, 7793–7804.